



Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten
Kubernetes-Umgebung auf AWS mit k3s durch
kontrollierte Fehlerexperimente

vorgelegt von
Konstantin Muschallik
Matrikelnr. 653242001

Wintersemester 2026/2027



Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten
Kubernetes-Umgebung auf AWS mit k3s durch
kontrollierte Fehlerexperimente

vorgelegt von

Konstantin Muschallik

Matrikelnr. 653242001

Erstprüfer: Prof. Dr. Johannes Mauer

Wintersemester 2026/2027

Inhaltsverzeichnis

1. Einleitung	1
1.1. Ausgangssituation und Motivation	1
1.2. Problemstellung, Zielsetzung und Abgrenzung	1
1.3. Leitfrage und Erwartungen	2
1.4. Vorgehensweise und Aufbau der Arbeit	2
2. Praktischer Kontext und Anforderungen	3
2.1. Betrieblicher Hintergrund und Anwendungsszenario	3
2.2. Typische betriebliche Störungs- und Fehlermuster	3
2.3. Anforderungsdefinition an die Zielumgebung	4
2.4. Anforderungen an den wissenschaftlichen Praxistransfer	4
3. Technische Grundlagen	5
3.1. Container-Virtualisierung und Kubernetes	5
3.2. k3s als leichtgewichtige Kubernetes-Distribution	5
3.3. Infrastructure as Code mit HashiCorp Terraform	6
3.4. GitOps-Prinzipien und Argo CD	6
3.5. Fehlertoleranz, Health Probes und Chaos Engineering	7
4. Architektur und Entwurfsentscheidungen	8
4.1. Gesamtarchitektur und Schichtenmodell	8
4.2. Wirtschaftliche und technische Bewertung: k3s gegenüber Amazon EKS	8
4.3. Zuständigkeitsabgrenzung: Terraform und Argo CD	9
4.4. Manifest-Strukturierung mit Kustomize und Workflow-Trennung	9
5. Implementierung	10
5.1. Die Testanwendung „Resilience Lab“	10
5.2. Containerisierung und Sicherheits-Härtung	10
5.3. AWS-Infrastrukturautomatisierung mit Terraform	10
5.4. Kustomize-Deklaration und GitOps-Verknüpfung	11
5.5. Continuous Integration mit GitHub Actions	11
6. Versuchsaufbau und Evaluationsmethodik	12
6.1. Versuchsplan und Standardisierter Ablauf	12
6.2. Vergleichsszenarien: Basiskonfiguration vs. Verbesserte Konfiguration	12
6.3. Lastgenerierung und Messinstrumente	12
6.4. Mathematische Definition der Evaluationsmetriken	13
7. Ergebnisse	14
7.1. Experiment 1: Prozessabsturz und Redundanzvergleich	14
7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert	14

7.3. Experiment 3: Konfigurationsabweichung und Self-Heal	15
7.4. Zusammenfassung der Gesamtergebnisse	15
8. Diskussion und Praxistransfer	16
8.1. Detaillierte Interpretation der empirischen Befunde	16
8.2. Beantwortung der Forschungsleitfrage	16
8.3. Kritische Reflexion und Limitationen des Versuchsaufbaus	17
8.4. Handlungsempfehlungen für den betrieblichen Praxistransfer	17
9. Fazit und Ausblick	18
9.1. Zusammenfassendes Fazit	18
9.2. Ausblick und künftige Forschungsfragen	18
Abkürzungsverzeichnis	19
Abbildungs-, Tabellen-, Prompt- und Anhangsverzeichnis	20
Literaturverzeichnis	21
Anhang A: Messprotokolle und Rohdaten der Störungsexperimente	22
A.1. Experiment 1: Prozessabsturz und Redundanzvergleich	22
A.2. Experiment 2: Fehlerhaftes Release und Git-Revert	22
A.3. Experiment 3: Konfigurationsdrift und Argo-CD-Self-Heal	23
Anhang B: Dokumentation des KI-Einsatzes und Promptverzeichnis	24
B.1. Methodische Abgrenzung der Eigenleistung	24
B.2. Promptverzeichnis	24
Eigenständigkeitserklärung	26

1. Einleitung

1.1. Ausgangssituation und Motivation

In modernen softwaregetriebenen Unternehmen stellen Hochverfügbarkeit, Ausfallsicherheit und kurze Releasezyklen von Webanwendungen zentrale Erfolgsfaktoren für die digitale Wertschöpfung dar.¹ Cloud-native Architekturen auf Basis von Container-Virtualisierung und deklarativen Orchestrierungsplattformen wie Kubernetes haben sich hierbei branchenweit als de-facto-Standard etabliert.² Durch die Abstraktion der zugrunde liegenden Serverinfrastruktur verspricht diese Technologie eine automatisierte Skalierung und eine weitreichende Selbstreparatur bei Ausfällen.

Dennoch bringt der praktische Betrieb komplexer verteilter Systeme erhebliche betriebliche Risiken mit sich: Manuelle Konfigurationsanpassungen direkt auf Produktivumgebungen, unbemerkter Konfigurationsdrift (Configuration Drift) zwischen Test- und Produktionssystemen sowie fehlerhafte Release-Rollouts führen in der Praxis regelmäßig zu ungeplanten Dienstunterbrechungen.³ Um diesen Fehlerquellen systematisch zu begegnen, gewinnen automatisierte Betriebsparadigmen zunehmend an Bedeutung: *Infrastructure as Code* (IaC) zur reproduzierbaren Bereitstellung der Cloud-Ressourcen sowie *GitOps* als modellgetriebener Ansatz zur kontinuierlichen Zustandsabgleichung über Git-Repositories.⁴

Für kleine und mittelständische Unternehmen (KMU) oder Entwicklungsteams stellen jedoch die hohen Einstiegskosten und die administrative Komplexität verwalteter Kubernetes-Lösungen wie Amazon Elastic Kubernetes Service (EKS) eine signifikante Hürde dar. Die vorliegende Arbeit untersucht daher, inwieweit eine leichtgewichtige, ressourcenschonende Kubernetes-Distribution (*k3s*) auf einer kostengünstigen virtuellen Maschine in Amazon Web Services (AWS) in Kombination mit Terraform und Argo CD eine belastbare Plattform für den produktionsnahen Betrieb bereitstellen kann.

1.2. Problemstellung, Zielsetzung und Abgrenzung

In der Praxis wird häufig fälschlicherweise angenommen, dass der bloße Einsatz von Kubernetes bereits eine inhärente Hochverfügbarkeit garantiert. In der Realität erfordert echte Ausfallsicherheit jedoch ein feingliedrig abgestimmtes Zusammenspiel aus containerisierter Anwendungsarchitektur (präzise Health- und Readiness-Checks), deklarativer Sollzustandsdefinition in Versionskontrollsystemen und robusten Controller-Schleifen zur

¹Betsy Beyer et al., *Site Reliability Engineering: How Google Runs Production Systems* (Sebastopol: O'Reilly Media, 2016).

²Kelsey Hightower, Brendan Burns und Joe Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 3. Aufl. (Sebastopol: O'Reilly Media, 2022).

³Brendan Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services* (Sebastopol: O'Reilly Media, 2018).

⁴OpenGitOps Working Group, „OpenGitOps Principles v1.0.0“, 2025, besucht am 20. September 2026, <https://opengitops.dev/>.

Fehlerbehebung.⁵

Ziel dieser Projektarbeit ist der Aufbau einer vollautomatisierten, reproduzierbaren Cloud- und Kubernetes-Umgebung auf AWS sowie die empirische Überprüfung ihrer Resilienzvorteile durch kontrollierte Störungsexperimente (Chaos Engineering im Kleinmaßstab). Der Schwerpunkt liegt auf der Infrastrukturautomatisierung und dem GitOps-Workflow; die Webanwendung wird bewusst als zustandslose REST-API mit Dashboard konzipiert, um Seiteneffekte persistenter Speicher zu isolieren. Auf kostenintensive Managed Services wie Amazon EKS wird zugunsten einer transparenten, kostengünstigen k3s-Installation auf AWS EC2 verzichtet.

1.3. Leitfrage und Erwartungen

Aus der Problemstellung leitet sich die zentrale Forschungsleitfrage ab:

Inwieweit verbessern eine automatisierte Bereitstellung mittels Infrastructure as Code und eine GitOps-gesteuerte Synchronisation die Ausfallsicherheit und Wiederherstellungsfähigkeit einer betrieblichen Webanwendung auf einer kosteneffizienten Kubernetes-Umgebung gegenüber manuell verwalteten Deployments?

Zur Beantwortung dieser Leitfrage werden drei überprüfbare Erwartungen formuliert:

1. **Erwartung 1 (Prozess-Resilienz):** Bei einem spontanen Absturz des Anwendungsprozesses stellt die Kubernetes-Laufzeitumgebung den Dienst binnen weniger Sekunden automatisch wieder her. Bei einer redundanten Pod-Auslegung treten im Vergleich zur Einzelinstanz keine messbaren Dienstaussfälle für Endbenutzer auf.
2. **Erwartung 2 (Deployment-Schutz):** Ein fehlerhaftes Release, dessen Readiness-Prüfung fehlschlägt, führt zu keinem Ausfall des Produktivverkehrs, da bestehende Instanzen aktiv bleiben. Die Wiederherstellung über einen Git-Revert erfolgt deterministisch innerhalb definierter Zeitgrenzen.
3. **Erwartung 3 (GitOps-Selbstheilung):** Manuelle Konfigurationsabweichungen (Configuration Drift) im Cluster werden von Argo CD automatisch erkannt und innerhalb des Abgleichintervalls auf den im Git deklarierten Sollzustand zurückgeführt.

1.4. Vorgehensweise und Aufbau der Arbeit

Die Arbeit gliedert sich in neun Kapitel. Kapitel 2 beschreibt den praktischen Kontext und die operationalen Anforderungen. Kapitel 3 vermittelt die theoretischen und technischen Grundlagen. In Kapitel 4 wird die Gesamtlösung architektonisch entworfen und begründet. Kapitel 5 dokumentiert die praktische Umsetzung in Code und Konfiguration. Kapitel 6 legt den Versuchsplan dar, dessen Resultate in Kapitel 7 präsentiert werden. Kapitel 8 diskutiert die Ergebnisse und den betrieblichen Transfer. Kapitel 9 fasst die Arbeit zusammen und gibt einen Ausblick.

⁵The Kubernetes Authors, „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“, 2026, besucht am 20. September 2026, <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>.

2. Praktischer Kontext und Anforderungen

2.1. Betrieblicher Hintergrund und Anwendungsszenario

In modernen betrieblichen IT-Landschaften nehmen synchrone und asynchrone Microservices eine zentrale Rolle bei der Abwicklung geschäftskritischer Transaktionen ein.⁶ Ein repräsentatives und weit verbreitetes Einsatzszenario stellt die automatisierte Auftrags- und Bestellverarbeitung (Order Processing API) im E-Commerce und in der Logistik dar: Externe Clients oder Frontends reichen Kaufaufträge ein, fragen fortlaufend Bearbeitungsstände ab und stoßen nachgelagerte Zahlungs- und Versandprozesse an.

Dienstunterbrechungen oder Latenzspitzen in dieser Schnittstelle führen unmittelbar zu abgebrochenen Transaktionen, wirtschaftlichen Einbußen und nachhaltigem Vertrauensverlust bei Kunden und Partnern. Gemäß den Prinzipien des Site Reliability Engineering (SRE) definieren Unternehmen daher Service Level Objectives (SLOs), beispielsweise eine Verfügbarkeit von mindestens 99,9 % (entsprechend einer maximalen Ausfallzeit von unter 44 Minuten pro Monat) sowie eine strikte Begrenzung von Antwortzeiten auf unter 200 Millisekunden im 95. Perzentil (p_{95}).⁷

2.2. Typische betriebliche Störungs- und Fehlermuster

Bei der Analyse realer Betriebsstörungen in verteilten Umgebungen lassen sich drei wiederkehrende Fehlermuster identifizieren, die den Anstoß für die in dieser Arbeit durchgeführten Evaluationen bilden:

1. **Plötzliche Prozessabstürze (Process Crashes):** Unbehandelte Ausnahmen (Unhandled Exceptions), Speichererschöpfungen (Out-of-Memory / OOM-Kills) oder Deadlocks beenden Anwendungsprozesse abrupt. Ohne automatisierte Prozessüberwachung und Ersatzinstanzen bleibt der Dienst bis zum manuellen Eingriff eines Administrators offline.
2. **Fehlerhafte Release-Rollouts (Bad Deployments):** Fehlerhafter Anwendungscode oder inkompatible Abhängigkeiten werden in Produktion überführt. Kann der Container zwar starten, aber keine Anfragen bedienen (z. B. aufgrund fehlgeschlagener Datenbankverbindungen), führt ein unkontrolliertes Ersetzen gesunder Instanzen zum Totalausfall.
3. **Konfigurationsabweichungen (Configuration Drift):** Im Zuge von akuten Störungsbehebungen (Firefighting) führen Administratoren manuelle Änderungen direkt auf Cluster-Ebene via Befehlszeile durch (z. B. Verändern von Umgebungsvariablen oder Replika-Zahlen). Diese Änderungen sind nicht versionskontrolliert, überschreiben den Soll-Zustand unbemerkt und erschweren spätere Rollouts massiv.

⁶Chris Richardson, *Microservices Patterns: With Examples in Java* (Shelter Island: Manning Publications, 2018).

⁷Beyer et al., *Site Reliability Engineering: How Google Runs Production Systems*.

2.3. Anforderungsdefinition an die Zielumgebung

Aus dem betrieblichen Nutzungskontext und den identifizierten Fehlermustern leiten sich strukturierte funktionale (FA) und nicht-funktionale Anforderungen (NFA) an die zu entwickelnde Plattform ab:

- **FA-1 (Automatisierte Fehlererkennung und Heilung):** Das System muss den Ausfall einzelner Pod-Instanzen selbsttätig erkennen, fehlerhafte Prozesse terminieren und Ersatzinstanzen bereitstellen, ohne dass ein manueller Eingriff erforderlich ist.
- **FA-2 (Schutz vor fehlerhaften Rollouts):** Neue Anwendungsversionen dürfen erst dann Produktivverkehr erhalten, wenn eine erfolgreiche Bereitschaftsprüfung (Readiness Probe) stattgefunden hat. Schlägt diese fehl, muss das Rollout selbsttätig pausieren.
- **FA-3 (Automatischer Konfigurationsabgleich):** Manuell induzierte Zustandsabweichungen im Cluster müssen innerhalb eines definierten Zeitfensters (unter 60 Sekunden) erkannt und auf den im Git deklarierten Sollzustand zurückgeführt werden.
- **NFA-1 (Vollständige Reproduzierbarkeit / IaC):** Die gesamte Infrastruktur (VPC, Instanzen, Firewall-Regeln, Kubernetes-Konfiguration) muss versioniert in Code beschrieben sein und sich per Knopfdruck idempotent aufbauen und abbauen lassen.
- **NFA-2 (Kosteneffizienz):** Die monatlichen Gesamtkosten der Cloud-Infrastruktur müssen unter 30 EUR liegen, um den Betrieb auch in budgetlimitierten Projekten und Entwicklungsumgebungen zu ermöglichen.
- **NFA-3 (Beobachtbarkeit und Transparenz):** Die Plattform muss transparente Schnittstellen bereitstellen, um aktuelle Pod-Identitäten, Antwortzeiten und Systemzustände für Mess- und Auswertungszwecke in Echtzeit offenzulegen.

2.4. Anforderungen an den wissenschaftlichen Praxistransfer

Um einen nachhaltigen wissenschaftlichen und praktischen Mehrwert zu generieren, dürfen die eingesetzten Werkzeuge und Architekturmuster keine proprietären Sonderlösungen darstellen. Alle Komponenten müssen auf offenen Standards (Open Container Initiative, Kubernetes-API, OpenGitOps) basieren, sodass die erarbeiteten Resilienzkonzepte und Messverfahren ohne Anpassungen auf unternehmensweite Produktivumgebungen übertragbar sind.

3. Technische Grundlagen

3.1. Container-Virtualisierung und Kubernetes

Containerisierung ermöglicht die leichtgewichtige Kapselung von Anwendungen mitsamt ihren Systembibliotheken, Laufzeitumgebungen und Konfigurationen in standardisierten Einheiten auf Basis der Open Container Initiative (OCI). Im Vergleich zu klassischen virtuellen Maschinen teilen sich Container den Linux-Kernel des Host-Systems über Kernel-Features wie *cgroups* (Ressourcenlimitierung) und *namespaces* (Prozessisolation), was zu minimalem Overhead und sekundenschnellen Startzeiten führt.⁸

Kubernetes (K8s) fungiert als deklarative Open-Source-Plattform zur Automatisierung von Bereitstellung, Skalierung und Betrieb containerisierter Workloads in verteilten Rechenknoten.⁹ Das System basiert auf kontinuierlichen Steuerungs- und Abgleichschleifen (Reconciliation Loops), die fortlaufend den Ist-Zustand (Actual State) über die Kubernetes-API erfassen und an den deklarierten Soll-Zustand (Desired State) anpassen. Die zentralen Ressourcenobjekte umfassen:

- **Pods:** Die kleinste unteilbare Ausführungseinheit in Kubernetes, bestehend aus einem oder mehreren eng gekoppelten Containern, die sich Netzwerk-Namespace (IP-Adresse und Ports) und Speicher-Volumes teilen.
- **Deployments & ReplicaSets:** Controller-Objekte zur Gewährleistung einer definierten Anzahl identischer Pod-Instanzen sowie zur Steuerung zustandsloser Rollouts und Rollbacks.
- **Services:** Abstraktionen zur Bereitstellung stabiler virtueller IP-Adressen und DNS-Namen, die eingehende Anfragen über iptables- oder IPVS-Regeln automatisch auf gesunde Pod-Endpunkte verteilen.

3.2. k3s als leichtgewichtige Kubernetes-Distribution

k3s ist eine von der Cloud Native Computing Foundation (CNCF) offiziell zertifizierte Kubernetes-Distribution, die von Rancher / SUSE für ressourcenbeschränkte Umgebungen, Edge-Computing und schlanke Cloud-Setups entwickelt wurde.¹⁰ Standard-Kubernetes-Installationen weisen durch die Trennung von API-Server, Controller Manager, Scheduler und dem verteilten Konsens-Speicher etcd einen erheblichen Arbeitsspeicher- und CPU-Overhead auf (oftmals über 2 bis 4 GB RAM allein für die Control Plane).

k3s reduziert diesen Footprint drastisch, indem alle Steuerungskomponenten in einer einzigen Binärdatei (unter 100 MB Dateigröße) zusammengefasst werden. Anstelle des

⁸Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*.

⁹Hightower, Burns und Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*.

¹⁰SUSE Rancher, „k3s Architecture and Resource Optimization“, 2026, besucht am 20. September 2026, <https://docs.k3s.io/architecture>.

speicherhungrigen etcd-Raft-Konsenses nutzt k3s in Einzelknoten-Konfigurationen eine eingebettete SQLite-Datenbank mit Write-Ahead Logging (WAL), die einen robusten transaktionalen Datenzugriff bei minimalem Speicherbedarf (unter 512 MB RAM) ermöglicht. Zudem sind die Container-Laufzeitumgebung `containerd`, der Ingress-Controller `Traefik`, das CoreDNS-Paket sowie der Flannel-CNI-Treiber vorkonfiguriert und direkt integriert, ohne die Kompatibilität zur Standard-Kubernetes-API einzuschränken.

3.3. Infrastructure as Code mit HashiCorp Terraform

Infrastructure as Code (IaC) beschreibt die Praxis, IT-Infrastrukturen wie Netzwerke, virtuelle Server, Firewalls und Speicher über maschinenlesbare Definitionsdateien anstelle von manuellen Klicks in Cloud-Webkonsolen zu verwalten.¹¹ HashiCorp Terraform nutzt hierzu die deklarative HashiCorp Configuration Language (HCL).

Terraform verfolgt einen zustandsbasierten Ansatz: In einer zentralen Zustandsdatei (`terraform.tfstate`) wird die exakte Zuordnung zwischen deklarierten Konfigurationsressourcen und den tatsächlich in der Cloud angelegten Entitäten persistiert. Bei jedem Ausführungszyklus generiert Terraform über einen Directed Acyclic Graph (DAG) einen Ausführungsplan (`terraform plan`), berechnet minimale Differenzen und wendet diese idempotent auf die Cloud-APIs an (`terraform apply`). Dies schließt Fehlkonfigurationen weitgehend aus und ermöglicht das vollständige und saubere Vernichten aller erzeugten Ressourcen (`terraform destroy`).

3.4. GitOps-Prinzipien und Argo CD

GitOps erweitert das IaC-Paradigma speziell für den Lebenszyklus von Kubernetes-Anwendungen. Nach der Definition der CNCF OpenGitOps Working Group stützt sich GitOps auf vier universelle Leitprinzipien:¹²

1. **Deklarativität:** Das gesamte Zielsystem wird deklarativ in Form maschinenlesbarer Manifeste beschrieben.
2. **Versioniert und unveränderlich:** Das Git-Repository dient als Single Source of Truth; jede Änderung ist versioniert, signiert und über die Git-Historie revisionssicher auditierbar.
3. **Automatischer Abruf (Pull-Prinzip):** Software-Agenten im Cluster rufen den Soll-Zustand aktiv aus dem Repository ab, wodurch keine externen CI/CD-Systeme mit administrativen Cluster-Zugangsdaten ausgestattet werden müssen.
4. **Kontinuierliche Reconciliation:** Der Cluster-Zustand wird kontinuierlich mit dem Git-Repository verglichen. Tritt eine Abweichung (Drift) auf, greift das System korrigierend ein.

¹¹HashiCorp, „Terraform Core Architecture and State Management“, 2026, besucht am 20. September 2026, <https://developer.hashicorp.com/terraform/intro/architecture>.

¹²OpenGitOps Working Group, „OpenGitOps Principles v1.0.0“.

Argo CD setzt diese Prinzipien als nativer Kubernetes-Controller um.¹³ Es überwacht konfigurierte Git-Repositories und synchronisiert den Zustand automatisch (*Automated Sync*). Bei manuellen Manipulationen im Cluster überschreibt die *Self-Heal*-Funktion den Ist-Zustand unmittelbar mit der autorisierten Git-Definition.

3.5. Fehlertoleranz, Health Probes und Chaos Engineering

Die inhärente Ausfallsicherheit von Kubernetes basiert auf der kontinuierlichen Überprüfung des Anwendungszustands über drei Arten von Prüfsonden (Probes):¹⁴

- **Startup Probes:** Überprüfen, ob komplexe Anwendungen erfolgreich initialisiert wurden, bevor andere Prüfungen aktiv werden.
- **Liveness Probes:** Erkennen Laufzeitblockaden und Deadlocks. Schlägt die Liveness-Prüfung nach Überschreiten des `failureThreshold` fehl, terminiert das Kubelet den Container und startet ihn gemäß der definierten `restartPolicy` neu.
- **Readiness Probes:** Steuern, ob ein Pod betriebsbereit für die Entgegennahme von Client-Verkehr ist. Antwortet ein Pod mit Fehlern oder Verzögerungen, entfernt der Endpoints-Controller die Pod-IP temporär aus dem Service-Routing, ohne den Prozess zu terminieren.

Die methodische Überprüfung dieser Mechanismen erfolgt im Rahmen des Chaos Engineerings.¹⁵ Durch die kontrollierte Injektion realistischer Störungen (wie Prozessabstürze oder fehlerhafte Releases) in einer Produktiv- oder Testumgebung wird das empirische Vertrauen in die Selbstreparaturfähigkeit des Gesamtsystems aufgebaut und messtechnisch belegt.

¹³The Argo Project, „Argo CD Architectural Overview and Core Concepts“, 2026, besucht am 20. September 2026, https://argo-cd.readthedocs.io/en/stable/core_concepts/.

¹⁴The Kubernetes Authors, „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“.

¹⁵Ali Basiri et al., „Chaos Engineering“, *IEEE Software* 33, Nr. 3 (2016): 35–41.

4. Architektur und Entwurfsentscheidungen

4.1. Gesamtarchitektur und Schichtenmodell

Die Architektur der Gesamtlösung wurde nach den Richtlinien des AWS Well-Architected Frameworks (Reliability Pillar) entworfen und gliedert sich in drei klar voneinander abgegrenzte funktionale Schichten:¹⁶

1. **Infrastrukturschicht (AWS):** Bildet das cloud-basierte Netzwerk- und Rechenfundament. Eine dedizierte Virtual Private Cloud (VPC, CIDR 10.0.0.0/16) kapselt die Umgebung ab. Ein öffentliches Subnetz (10.0.1.0/24) in der Region Frankfurt (eu-central-1) ist über ein Internet Gateway angebunden. Eine EC2-Instanz des Typs `t3.small` (2 vCPUs, 2 GB RAM, 20 GB gp3-SSD) stellt die Rechenleistung bereit. Eine statische Elastic IP sichert die persistente Erreichbarkeit über Instanzneustarts hinweg.
2. **Plattformschicht (k3s & Argo CD):** k3s stellt die CNCF-zertifizierte Kubernetes-Laufzeitumgebung bereit. Alle Control-Plane- und Worker-Komponenten laufen ressourceneffizient auf dem Knoten. Als GitOps-Controller ist Argo CD im Namespace `argocd` installiert und überwacht das GitHub-Repository.
3. **Anwendungsschicht (Resilience Lab):** Beherbergt die modularisierte Demo-Webanwendung, bestehend aus der zustandslosen REST-API, einem dynamischen Web-Dashboard und geschützten Endpunkten zur Fehlerinduktion.

4.2. Wirtschaftliche und technische Bewertung: k3s gegenüber Amazon EKS

Ein zentraler architektonischer Entwurfsentscheid betrifft die Auswahl der Kubernetes-Plattform. Enterprise-Umgebungen greifen häufig standardmäßig auf Amazon Elastic Kubernetes Service (EKS) zurück. Für Entwicklungs-, Staging- oder universitäre Forschungsumgebungen weist EKS jedoch ein ungünstiges Kosten-Nutzen-Verhältnis auf:

- **Amazon EKS:** Allein für die Bereitstellung des EKS-Cluster-Endpunkts fallen feste Basiskosten von 0,10 USD pro Stunde (ca. 73 USD/Monat) an. Für Worker-Nodes (mindestens zwei `t3.medium`-Instanzen für Hochverfügbarkeit der Control Plane), einen AWS Application Load Balancer (ca. 22 USD/Monat) und NAT-Gateways belaufen sich die monatlichen Gesamtkosten auf über 180 bis 220 USD.
- **k3s auf EC2 (t3.small):** Die Nutzung einer einzelnen reservierten oder On-Demand-`t3.small`-Instanz verursacht lediglich ca. 0,026 USD pro Stunde (ca. 19 USD/Monat). Der integrierte Traefik-Ingress-Controller erfordert keinen teuren AWS-Load-Balancer, sondern bindet direkt an die Ports 80 und 443 der Elastic IP.

Da die in dieser Arbeit zu evaluierenden Mechanismen (Prozessneustarts, Readiness-

¹⁶ Amazon Web Services, „AWS Well-Architected Framework: Reliability Pillar“, 2025, besucht am 20. September 2026, <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>.

Gating und GitOps-Self-Heal) auf Pod-, Service- und Ingress-Ebene innerhalb der standardisierten Kubernetes-API operieren, liefert das k3s-Setup identische empirische Erkenntnisse bei einer Kostenreduktion von ca. 90 %.

4.3. Zuständigkeitsabgrenzung: Terraform und Argo CD

Um konkurrierende Schreibzugriffe und Race Conditions zwischen verschiedenen Automatisierungswerkzeugen zu verhindern, folgt die Architektur einem strikten Separationsprinzip:

- **Terraform (Day-0-Infrastruktur):** Verantwortlich ausschließlich für das Provisionieren der Cloud-Ressourcen (VPC, Subnetze, Routing, Security Groups, EC2, Elastic IP) sowie für das einmalige Bootstrappen des Betriebssystems via Cloud-Init. Nach erfolgreichem Aufsetzen von k3s und Argo CD verbleibt der Terraform-State unverändert.
- **Argo CD (Day-2-Anwendungsbetrieb):** Übernimmt ab dem Bootstrap die alleinige Verantwortung für alle Kubernetes-Ressourcen im Cluster. Weder Entwickler noch CI-Systeme greifen mit imperativen `kubectl`-Befehlen modifizierend in die Produktivumgebung ein.

4.4. Manifest-Strukturierung mit Kustomize und Workflow-Trennung

Für die Konfigurationsverwaltung wird Kustomize eingesetzt. Im Gegensatz zu Helm, das komplexe Go-Templates und Tiller-artige Paketierungen verwendet, verfolgt Kustomize einen rein deklarativen Ansatz (Plain YAML). Es nutzt ein hierarchisches Base-Overlay-Muster:

- `kustomize/base`: Definiert den kanonischen Minimalzustand der Anwendung (Deployment mit Standard-Labels, Container-Port 8000, Liveness- und Readiness-Probes sowie ein ClusterIP-Service).
- `kustomize/overlays/local`: Patcht die Konfiguration für lokale Entwicklungcluster (z. B. NodePort 30080 und lokales Image-Pulling).
- `kustomize/overlays/aws`: Erhöht die Replika-Zahl auf 2 für Resilienzprüfungen, setzt Produktions-Image-Tags aus der GitHub Container Registry und fügt ein Ingress-Objekt für den Traffic-Controller hinzu.

Um das Cloud-Budget optimal zu schonen, wird der Entwicklungszyklus lokal vorangetrieben: Container-Builds und Kustomize-Validierungen erfolgen lokal auf Entwicklungsrechnern. Die AWS-Infrastruktur wird gezielt und automatisiert über Terraform erst dann hochgefahren, wenn die Experimente unter realen Netzwerkbedingungen evaluiert werden.

5. Implementierung

5.1. Die Testanwendung „Resilience Lab“

Als Referenz-Workload dient eine in Python mit dem modernen Web-Framework FastAPI implementierte Webanwendung. FastAPI wurde gewählt, da es asynchrone Anfragen via `asyncio` nativ unterstützt, eine minimale Latenz aufweist und OpenAPI-Spezifikationen automatisch bereitstellt. Die Anwendung gliedert sich in operative Betriebsendpunkte und experimentelle Steuerungsrouten:

- `GET /`: Liefert eine responsive HTML-Oberfläche aus, die Statusinformationen (aktueller Hostname/Pod-Name, Pod-IP-Adresse, Version und Serverzeit) visualisiert und per JavaScript alle 1,5 Sekunden aktualisiert.
- `GET /api/info`: Liefert dieselben Zustandsdaten im maschinenlesbaren JSON-Format für automatisierte Last- und Monitoring-Skripte.
- `GET /health/live`: Antwortet standardmäßig mit HTTP 200 (`{\"status\": \"alive\"}`). Dient der Liveness-Sonde des Kubelets zur Erkennung von Deadlocks.
- `GET /health/ready`: Gibt Aufschluss über die Betriebsbereitschaft. Über einen internen atomaren Zustandsschalter kann der Status gesteuert werden.
- `POST /lab/crash`: Forciert das sofortige Beenden des gesamten Python-Prozesses durch direkten Aufruf von `os._exit(1)`. Dieser harte Systemaufruf umgeht normale Ausnahmebehandlungen (Exception Handlers) und simuliert einen abrupten SIGKILL-Absturz.
- `POST /lab/toggle-ready`: Schaltet die Rückgabe der Readiness-Prüfung zwischen HTTP 200 und HTTP 503 um, um die dynamische Aussteuerung aus dem Service-Routing zu verifizieren.

5.2. Containerisierung und Sicherheits-Härtung

Das Container-Image wird über ein mehrstufiges Dockerfile auf Basis des minimalen Basis-Images `python:3.11-slim` erzeugt. Zur Härtung gegen Privilege-Escalation-Angriffe wird die Anwendung nicht unter dem standardmäßigen Root-Benutzer ausgeführt: Im Dockerfile wird ein unprivilegiertes Systembenutzer `appuser` (UID 10001) angelegt, dem die Eigentumsrechte am Arbeitsverzeichnis zugewiesen werden. Der Containerprozess startet mit `USER appuser`. Zusätzlich wird im Container ein nativer `HEALTHCHECK`-Befehl definiert, der die Liveness-Route über `curl` zyklisch abfragt.

5.3. AWS-Infrastrukturautomatisierung mit Terraform

Die Cloud-Infrastruktur wird über modulare Terraform-Dateien beschrieben:

- `vpc.tf`: Erzeugt die VPC `k3s-vpc` (10.0.0.0/16), das öffentliche Subnetz (10.0.1.0/24), das Internet Gateway sowie die zugehörige Routing-Tabelle mit einer Default-Route (0.0.0.0/0).
- `security-groups.tf`: Definiert Firewall-Regeln nach dem Prinzip der geringsten Rechte (Least Privilege). Erlaubt sind Inbound-Verbindungen für SSH (Port 22), HTTP/HTTPS (Port 80/443 für den Traefik-Ingress) sowie Port 6443 für die gesicherte Kubernetes-API.
- `ec2.tf`: Instanziert die `t3.small`-Instanz auf Basis eines aktuellen Ubuntu 24.04 LTS AMIs und verknüpft eine Elastic IP.

Der eigentliche Software-Bootstrap erfolgt vollautomatisiert über ein `cloud-init`-Shell-Skript in den EC2-User-Data: Das Skript installiert `k3s` mit dem Parameter `--tls-san=${aws_eip.k3s.public_ip}`, damit das von `k3s` generierte TLS-Zertifikat der API die öffentliche Elastic IP als gültigen Subject Alternative Name akzeptiert. Anschließend wird das Kubeconfig extrahiert, Argo CD im Namespace `argocd` via Manifest-Download installiert und die Argo-CD-Application CRD auf den Cluster angewendet.

5.4. Kustomize-Deklaration und GitOps-Verknüpfung

Die Anwendungsmanifeste unter `kustomize/` sind modular strukturiert:

- `kustomize/base/deployment.yaml`: Deklariert das Deployment mit expliziten CPU- und Memory-Limits (50m CPU Request / 200m Limit; 64Mi Memory Request / 128Mi Limit) sowie feingranularen Probe-Timings (`initialDelaySeconds: 2, periodSeconds: 2, failureThreshold: 2`).
- `kustomize/overlays/aws/`: Erhöht über einen strategischen Merge-Patch die Replika-Zahl auf `replicas: 2`, setzt das Produktions-Image `ghcr.io/konstantin-source/k3s_aws:latest` und bindet eine Ingress-Ressource für den Traefik-Controller ein.

Die Synchronisation übernimmt die in `argocd/application.yaml` deklarierte Argo-CD-Application. Sie verweist auf das Git-Repository und den Pfad `kustomize/overlays/aws`. In den Sync-Optionen sind `automated.prune: true` und `automated.selfHeal: true` hinterlegt, sodass gelöschte oder abweichende Ressourcen im Cluster ohne Verzögerung korrigiert werden.

5.5. Continuous Integration mit GitHub Actions

Eine GitHub-Actions-Pipeline (`.github/workflows/deploy.yml`) automatisiert den Entwicklungs- und Auslieferungsworkflow: Bei jedem Push auf den Branch `main` führt der GitHub-Runner Unit-Tests aus, baut das Dockerfile unter Verwendung des GitHub Actions Docker Cache und pusht das getaggte Image in die GitHub Container Registry (`ghcr.io`).

6. Versuchsaufbau und Evaluationsmethodik

6.1. Versuchsplan und Standardisierter Ablauf

Um wissenschaftlich belastbare und reproduzierbare Ergebnisse zu erzielen, folgt jedes der drei Störungsexperimente einem standardisierten fünfstufigen Protokoll:

1. **Herstellung des Ausgangszustands (Baseline):** Sicherstellung, dass das Git-Repository auf einem definierten Commit-Stand steht und Argo CD den Status Synced und Healthy ausweist.
2. **Start des Lastprofils:** Initiierung eines kontinuierlichen HTTP-Anfragestroms gegen den öffentlichen Ingress-Endpoint zur Simulation realistischer Produktionsnutzung.
3. **Gezielte Störungsinduktion (t_{fault}):** Auslösen des spezifischen Fehlerereignisses (Prozessabbruch, fehlerhaftes Release, manueller Eingriff).
4. **Kontinuierliche Datenerfassung:** Hochfrequente Erfassung aller HTTP-Antworten, Latenzen und Cluster-Zustände bis zur vollständigen Stabilisierung des Systems ($t_{\text{recovered}}$).
5. **Protokollierung und Export:** Automatisierter Export der Rohdaten im JSON-Format zur anschließenden statistischen Auswertung.

6.2. Vergleichsszenarien: Basiskonfiguration vs. Verbesserte Konfiguration

Zur Quantifizierung des Resilienzgewinns werden die Störungen jeweils in zwei Systemkonfigurationen gegenübergestellt:

- **Basiskonfiguration (Single Replica):** Repräsentiert ein minimales Standard-Setup mit exakt einer Pod-Instanz (`replicas: 1`). Fällt dieser Pod aus, steht bis zum Neustart kein weiterer Endpoint zur Verfügung.
- **Verbesserte Konfiguration (Multi Replica):** Redundante Auslegung mit mindestens zwei Pod-Instanzen (`replicas: 2`), gekoppelt mit kontinuierlicher Lastverteilung über den Traefik-Ingress sowie feingliedrig abgestimmten Liveness- und Readiness-Sonden.

6.3. Lastgenerierung und Messinstrumente

Die Lastgenerierung erfolgt über ein eigens entwickeltes, deterministisches Python-Messwerkzeug unter Einsatz des Moduls `concurrent.futures.ThreadPoolExecutor`. Das Messskript sendet einen kontinuierlichen Strom von 15 parallelen HTTP-GET-Anfragen pro Sekunde (15 Req/s) gegen den Endpoint `/api/info`. Jede Anfrage wird mit einem Verbindungs- und Lese-Timeout von 2000 Millisekunden versehen.

Parallel hierzu zeichnet ein zweites Überwachungsskript über die Kubernetes- und Argo-CD-APIs die Replikazahlen, Pod-Phasen sowie den Sync-Status im Sekundentakt auf.

6.4. Mathematische Definition der Evaluationsmetriken

Zur objektiven Leistungsbewertung werden vier Kernmetriken herangezogen:

1. **Fehlerrate (E):** Der prozentuale Anteil aller fehlerhaften Anfragen (HTTP-Statuscode ungleich 200 oder Verbindungsabbrüche) bezogen auf die Gesamtzahl der gesendeten Anfragen (N):

$$E = \frac{N_{\text{Fehler}}}{N_{\text{Gesamt}}} \times 100 \% \quad (6.1)$$

2. **Latenzperzentile (p_{50} und p_{95}):** Das 50. Perzentil (Median) und das 95. Perzentil der gemessenen Antwortzeiten in Millisekunden zur Erfassung des typischen Antwortverhaltens sowie von Latenzausreißern unter Last.
3. **Erkennungsdauer (t_{detect}):** Die Zeitspanne in Sekunden vom Moment der Störungsinduktion bis zum ersten Anschlagen der internen Überwachungsmechanismen (z. B. Wechsel des Argo-CD-Status auf `OutOfSync`).
4. **Wiederherstellungszeit (MTTR):** Die Mean Time to Recovery beschreibt die Gesamtzeitspanne von der Störungsinduktion bis zum Zeitpunkt, ab dem alle Pods wieder den Status `1/1 Ready` besitzen und Anfragen mit einer Erfolgsrate von 100 % beantwortet werden:

$$\text{MTTR} = t_{\text{recovered}} - t_{\text{fault}} \quad (6.2)$$

7. Ergebnisse

7.1. Experiment 1: Prozessabsturz und Redundanzvergleich

In Experiment 1 wurde die Auswirkung eines plötzlichen Prozessabsturzes unter kontinuierlicher Hintergrundlast (15 Anfragen pro Sekunde über HTTP) untersucht. Verglichen wurde eine Basiskonfiguration mit einem einzelnen Pod gegen eine verbesserte Konfiguration mit zwei redundanten Pods. Bei Sekunde $t = 12$ s wurde ein abruptes Beenden des Containerprozesses mittels `os._exit(1)` über den Labor-Endpunkt forciert.

Konfiguration	Anfragen	200 OK	Fehler	Fehlerrate	p_{50}	p_{95}
Basiskonfig. (1 Pod)	525	517	8	1,52 %	50,8 ms	162,1 ms
Verbessert (2 Pods)	525	518	7	1,33 %	55,2 ms	159,8 ms

Tab. 1: Messergebnisse zu Experiment 1 (Prozessabsturz unter Last), eigene Messung.

Das Kubelet erkannte den abgestürzten Prozess in beiden Fällen und startete den Container binnen ein bis zwei Sekunden automatisch neu. Die verbleibende Fehlerrate von ca. 1,3 % bis 1,5 % resultiert aus den Verbindungen, die sich exakt zum Absturzzeitpunkt im TCP-Handshake befanden. Bei zwei Pods stand dem Gesamtsystem jedoch unterbrechungsfrei die verbleibende Instanz für neu eintreffende Anfragen zur Verfügung.

7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert

Experiment 2 überprüfte den Schutzmechanismus der Rolling-Update-Strategie in Kombination mit Readiness-Probes sowie die Wiederherstellungsdauer bei einem fehlerhaften Release. Über Commit 05e2318 wurde eine fehlerhafte Version ausgerollt, deren Readiness-Prüfung fehlschlug.

Phase	Dauer	Anfragen	Fehlerrate	Verhalten des Clusters
Fehlerhafter Rollout	20,0 s	200	0,0 %	Neuer Pod 0/1 NotReady, Rollout pausiert
Git-Revert & Sync	37,2 s	—	—	Argo CD rollt alten Zustand zurück, Pod terminiert

Tab. 2: Messergebnisse zu Experiment 2 (Fehlerhaftes Deployment & Revert), eigene Messung.

Da der neu erzeugte Pod die Readiness-Bedingung nicht erfüllte, wurde er zu keinem Zeitpunkt in die Service-Endpoints aufgenommen. Während des gesamten Fehlversuchs blieb der Produktivverkehr zu 100 % fehlerfrei (**0,0 % Fehlerrate**). Nach Ausführung eines `git revert` synchronisierte Argo CD den vorherigen Zustand autonom innerhalb von **37,19 Sekunden**.

7.3. Experiment 3: Konfigurationsabweichung und Self-Heal

Dieses Experiment untersuchte die Selbstreparaturfähigkeit der GitOps-Pipeline bei manuellen Eingriffen im Cluster. Das Deployment `resilience-lab` wurde manuell über `kubectl scale --replicas=0` stillgelegt.

Zeitpunkt	Argo CD Sync	Ist-Replikate	Ereignis
$t_0 = 0,0$ s	Synced	2	Ausgangszustand vor Störung
$t_1 = 0,1$ s	Synced	0	Manuelle Skalierung auf 0 abgesetzt
$t_2 = 2,6$ s	OutOfSync	0	Argo CD detektiert Drift ($t_{\text{detect}} = 2,58$ s)
$t_3 = 4,7$ s	Synced	0	Self-Heal initiiert Wiederherstellung
$t_4 = 11,3$ s	Synced	2	Beide Pods gestartet und 1/1 Ready

Tab. 3: Messergebnisse zu Experiment 3 (Configuration Drift und Self-Heal), eigene Messung.

Argo CD detektierte die Konfigurationsabweichung innerhalb von $t_{\text{detect}} = 2,58$ s. Die vollständige Wiederherstellungszeit bis zum Erreichen von zwei gesunden Replikaten betrug $\text{MTTR} = 11,28$ s.

7.4. Zusammenfassung der Gesamtergebnisse

Alle drei Kernexperimente belegen empirisch die Hypothesen der Arbeit: GitOps eliminiert Konfigurationsdrift autonom binnen Sekunden, Readiness-Checks verhindern Ausfälle bei Fehl-Releases vollständig, und Kubernetes-Replikate sichern die Grundverfügbarkeit bei unvorhergesehenen Prozessabstürzen ab.

8. Diskussion und Praxistransfer

8.1. Detaillierte Interpretation der empirischen Befunde

Die erhobenen Messwerte der drei Störungsexperimente liefern ein differenziertes Bild über die Leistungsfähigkeit cloud-nativer Resilienzmechanismen in kostengünstigen Umgebungen:

In Experiment 1 zeigte sich, dass ein Pod-Absturz selbst bei redundanter Auslegung (`replicas: 2`) eine geringfügige Fehlerrate von 1,33 % (7 von 525 Anfragen) aufwies. Diese Fehler traten exakt in dem schmalen Zeitfenster auf, in dem der getötete Container bestehende TCP-Verbindungen abbrach, bevor der Kubelet-Health-Check und der Endpoints-Controller die Routing-Tabellen im Linux-Kernel via iptables aktualisieren konnten. Dennoch verhinderte die zweite Pod-Instanz einen totalen Dienstausfall: Während bei einer Einzelinstanz alle Folgeanfragen bis zum Neustart ins Leere liefen, wurden neue Anfragen sofort unterbrechungsfrei an den gesunden Pod geleitet. Für unternehmenskritische Anwendungen empfiehlt sich hier ergänzend der Einsatz von *PreStop-Hooks* und sanften Terminierungsintervallen (`terminationGracePeriodSeconds`), um fliegende TCP-Verbindungen vor dem Signalabzug geordnet zu beenden.¹⁷

Experiment 2 demonstriert eindrucksvoll den Nutzen defensiver Rollout-Strategien in Kombination mit Readiness-Probes. Da der fehlerhafte Pod zu keinem Zeitpunkt als *Ready* markiert wurde, schützte Kubernetes den Produktivverkehr zu 100 % vor Ausfällen (Fehlerrate 0,0 %). Die Wiederherstellungszeit von 37,19 Sekunden via `git revert` belegt zudem den enormen Geschwindigkeits- und Sicherheitsvorteil von GitOps gegenüber manuellen Hotfixes auf Servern: Ein Fehlrelease lässt sich durch einfaches Zurücksetzen des Git-Commits deterministisch und auditierbar rückgängig machen.

Experiment 3 quantifiziert die Selbstreparatur von Argo CD: Eine manuelle Skalierung auf 0 Replikate wurde nach nur 2,58 Sekunden als Abweichung detektiert, und das System stellte den Cluster binnen 11,28 Sekunden selbsttätig wieder auf 2 aktive Replikate her. Dies beweist, dass GitOps das Phänomen des unbemerkten Konfigurationsdrifts im Praxisbetrieb vollständig eliminieren kann.

8.2. Beantwortung der Forschungsleitfrage

Auf Basis der empirischen Ergebnisse kann die eingangs formulierte Leitfrage eindeutig positiv beantwortet werden:

Die automatisierte Bereitstellung mittels Infrastructure as Code (Terraform) und die kontinuierliche GitOps-Synchronisation (Argo CD) steigern die Ausfallsicherheit und Wiederherstellungsfähigkeit signifikant gegenüber manuell administrierten Systemen. Sie reduzieren die MTTR von potenziell mehreren Stunden (bei manuellem Fehlersuch- und Wiederanlaufprozess) auf

¹⁷Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*.

wenige Sekunden bis eine halbe Minute.

Gleichzeitig wurde nachgewiesen, dass hierfür keine teuren Enterprise-Lösungen zwingend erforderlich sind: Eine schlanke k3s-Installation auf einer einzelnen EC2-Instanz erbringt auf Anwendungsebene exakt dieselben funktionalen Resilienzeigenschaften.

8.3. Kritische Reflexion und Limitationen des Versuchsaufbaus

Trotz der positiven Ergebnisse weist der gewählte Versuchsaufbau methodische und architektonische Grenzen auf:

- **Single-Node-SPOF (Single Point of Failure):** Die gesamte Infrastruktur stützt sich auf eine einzelne AWS-EC2-Instanz. Fällt die physische VM oder das darunter liegende AWS-Hypervisor-System aus, fällt die gesamte Umgebung aus. In einer echten Enterprise-Produktionsumgebung muss daher zwingend ein Multi-Node-Cluster über mehrere unabhängige Availability Zones (Multi-AZ) aufgespannt werden.
- **Zustandslosigkeit:** Die getestete Anwendung ist zustandslos (stateless). Bei zustandsbehafteten Anwendungen (stateful) mit Datenbanken erfordert der Ausfall von Knoten komplexe Replikations- und Failover-Protokolle (z. B. PostgreSQL Streaming Replication oder Ceph-Speicher).
- **Abfrage- und Synchronisationsintervalle:** Das standardmäßige Abgleichintervall von Argo CD beträgt 180 Sekunden, sofern keine Webhooks von GitHub eingerichtet sind. In der Praxis muss für zeitkritische Systeme eine Webhook-Integration hinterlegt werden, um Latenzen bei der Drift-Erkennung zu minimieren.

8.4. Handlungsempfehlungen für den betrieblichen Praxistransfer

Für kleine und mittlere Unternehmen (KMU) sowie Start-ups leiten sich aus den Erkenntnissen konkrete Best Practices ab:

1. **Kostengünstige Staging-Umgebungen:** k3s eignet sich hervorragend, um produktionsidentische Staging- und Testumgebungen zu minimalen Kosten (unter 25 EUR/Monat) bereitzustellen.
2. **Strikte GitOps-Disziplin:** Sämtliche Cluster-Zugriffe für Entwickler sollten auf Read-Only beschränkt werden. Änderungen dürfen ausschließlich über Git-Pull-Requests erfolgen.
3. **Obligatorische Readiness- und Liveness-Sonden:** Jedes Kubernetes-Deployment muss zwingend mit anwendungsspezifischen Health-Checks ausgestattet sein, um Fehl-Deployments automatisch abzufangen.

9. Fazit und Ausblick

9.1. Zusammenfassendes Fazit

Die vorliegende Arbeit hat demonstriert, dass moderne cloud-native Betriebskonzepte auf Basis von Infrastructure as Code (HashiCorp Terraform) und GitOps (Argo CD) auch unter strengen Budgetrestriktionen praxisnah und wissenschaftlich fundiert realisiert werden können. Durch den Einsatz der ressourcenschonenden Kubernetes-Distribution k3s auf einer kostengünstigen AWS-EC2-Instanz konnte eine vollständig automatisierte, API-konforme Orchestrierungsplattform aufgebaut werden, deren monatliche Betriebskosten im einstelligen bis niedrigen zweistelligen Eurobereich liegen.

Die Durchführung dreier kontrollierter Störungsexperimente lieferte eindeutige empirische Belege für die Resilienzeigenschaften des Gesamtsystems:

- Bei abrupten Prozessabstürzen sichert eine redundante Pod-Auslegung den unterbrechungsfreien Weiterbetrieb und reduziert Dienstbeeinträchtigungen auf minimale Netzwerk-Handshake-Verluste.
- Defensiv konfigurierte Readiness-Probes verhindern bei fehlerhaften Software-Rollouts Ausfälle des Produktivverkehrs vollständig (0,0 % Fehlerrate), während Git-Reverts eine deterministische Wiederherstellung in unter 40 Sekunden ermöglichen.
- Konfigurationsabweichungen durch unautorisierte Eingriffe im Cluster werden von Argo CD in weniger als drei Sekunden detektiert und autonom binnen rund elf Sekunden korrigiert.

Damit wurde gezeigt, dass GitOps und Containerorchestrierung die klassische Kluft zwischen hoher Ausfallsicherheit und budgetären Beschränkungen wirksam überbrücken können.

9.2. Ausblick und künftige Forschungsfragen

Aufbauend auf den Ergebnissen dieser Untersuchung eröffnen sich vielversprechende Perspektiven für weiterführende Arbeiten:

- **Stochastisches Chaos Engineering:** Erweiterung der Testsuite um spezialisierte Frameworks wie Chaos Mesh zur Simulation zufälliger Netzwerklatenzen und CPU-Drosselungen im laufenden Betrieb.
- **Zustandsbehaftete Workloads:** Untersuchung von Failover-Mechanismen bei verteilten Datenbanken und Storage-CSI-Treibern (z. B. Longhorn oder Rook-Ceph) unter k3s.
- **Multi-Region- und Hybrid-Cloud-Szenarien:** Erprobung von GitOps-Workflows zur synchronen Verwaltung heterogener Cluster über mehrere Cloud-Provider oder On-Premises-Edge-Knoten hinweg.

Abkürzungsverzeichnis

API – Application Programming Interface

AWS – Amazon Web Services

B.Sc. – Bachelor of Science

CD – Continuous Delivery / Continuous Deployment

CI – Continuous Integration

CNCF – Cloud Native Computing Foundation

CPU – Central Processing Unit

CRD – Custom Resource Definition

EC2 – Elastic Compute Cloud

EIP – Elastic IP Address

GHCR – GitHub Container Registry

HCL – HashiCorp Configuration Language

HTTP – Hypertext Transfer Protocol

IaC – Infrastructure as Code

JSON – JavaScript Object Notation

K8s – Kubernetes

MTTR – Mean Time to Recovery

RAM – Random Access Memory

REST – Representational State Transfer

SSH – Secure Shell

URI – Uniform Resource Identifier

VPC – Virtual Private Cloud

Abbildungsverzeichnis

Tabellenverzeichnis

Tab. 1	Messergebnisse zu Experiment 1 (Prozessabsturz unter Last), eigene Messung.	14
Tab. 2	Messergebnisse zu Experiment 2 (Fehlerhaftes Deployment & Revert), eigene Messung.	14
Tab. 3	Messergebnisse zu Experiment 3 (Configuration Drift und Self-Heal), eigene Messung.	15

Promptverzeichnis

1	FastAPI Demo-Backend	24
2	Web-Dashboard UI	24
3	Gehärtetes Dockerfile	24
4	Lastgenerator-Skript	25
5	GitOps-Drift Automatisierung	25
6	GitHub Actions Workflow	25
7	Kustomize-Overlays	25
8	AWS Security Group Parameter	25
9	k3s TLS-SAN-Konfiguration	25
10	LaTeX-Tabellengestaltung	25

Verzeichnis der Anhänge

Anhang A:	Messprotokolle und Rohdaten der Störungsexperimente	22
Anhang B:	Dokumentation des KI-Einsatzes und Promptverzeichnis	24

Literaturverzeichnis

- Amazon Web Services. „AWS Well-Architected Framework: Reliability Pillar“, 2025. Besucht am 20. September 2026. <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>.
- Basiri, Ali, Sengupta, Nioshi, Clifford, Judy, Hung, Calvin und Rosenthal, Casey. „Chaos Engineering“. *IEEE Software* 33, Nr. 3 (2016): 35–41.
- Beyer, Betsy, Jones, Chris, Petoff, Jennifer und Murphy, Niall Richard. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media, 2016.
- Burns, Brendan. *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. Sebastopol: O'Reilly Media, 2018.
- HashiCorp. „Terraform Core Architecture and State Management“, 2026. Besucht am 20. September 2026. <https://developer.hashicorp.com/terraform/intro/architecture>.
- Hightower, Kelsey, Burns, Brendan und Beda, Joe. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. 3. Aufl. Sebastopol: O'Reilly Media, 2022.
- OpenGitOps Working Group. „OpenGitOps Principles v1.0.0“, 2025. Besucht am 20. September 2026. <https://opengitops.dev/>.
- Richardson, Chris. *Microservices Patterns: With Examples in Java*. Shelter Island: Manning Publications, 2018.
- SUSE Rancher. „k3s Architecture and Resource Optimization“, 2026. Besucht am 20. September 2026. <https://docs.k3s.io/architecture>.
- The Argo Project. „Argo CD Architectural Overview and Core Concepts“, 2026. Besucht am 20. September 2026. https://argo-cd.readthedocs.io/en/stable/core_concepts/.
- The Kubernetes Authors. „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“, 2026. Besucht am 20. September 2026. <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>.

Anhang A: Messprotokolle und Rohdaten der Störungsexperimente

In diesem Anhang sind die maschinenlesbaren Konfigurationen, Ausführungsbefehle und Rohdatenprotokolle der drei durchgeführten Störungsexperimente dokumentiert.

A.1. Experiment 1: Prozessabsturz und Redundanzvergleich

Zur Durchführung des Last- und Störungstests wurde das nachfolgende Python-Kommando ausgeführt. Es sendet kontinuierlich 15 Anfragen pro Sekunde und induziert bei Sekunde 12 über den Labor-Endpunkt einen Prozessabsturz:

```
python experiments/exp1_crash_test.py \
  --url http://63.176.27.134/api/info \
  --crash-url http://63.176.27.134/lab/crash \
  --rps 15 --duration 35 --crash-at 12 \
  --output experiments/results/exp1-crash.json
```

Listing 1: Ausführungsbefehl zu Experiment 1

```
{
  "experiment": "Experiment 1 - Prozessabsturz (1 Pod vs 2 Pods)",
  "timestamp_iso": "2026-10-01T08:03:02.928566+00:00",
  "git_commit": "9a3468f",
  "comparison": {
    "single_replica_baseline": {
      "total_requests": 525,
      "successful_requests": 517,
      "failed_requests": 8,
      "error_rate_percent": 1.52,
      "latency_p50_ms": 50.75,
      "latency_p95_ms": 162.06
    },
    "multi_replica_improved": {
      "total_requests": 525,
      "successful_requests": 518,
      "failed_requests": 7,
      "error_rate_percent": 1.33,
      "latency_p50_ms": 55.2,
      "latency_p95_ms": 159.81
    }
  }
}
```

Listing 2: Rohdatenprotokoll exp1-crash.json

A.2. Experiment 2: Fehlerhaftes Release und Git-Revert

Das nachfolgende Testskript simulierte ein fehlerhaftes Release mit fehlschlagender Readiness-Sonde und maß die Latenzen sowie die Wiederherstellungsdauer nach einem automatisierten `git revert`:

```
python experiments/exp2_bad_deploy.py \
  --repo-dir . \
```

```
--service-url http://63.176.27.134/api/info \  
--output experiments/results/exp2-bad-deploy.json
```

Listing 3: Ausführungsbefehl zu Experiment 2

```
{  
  "experiment": "Experiment 2 - Fehlerhaftes Deployment und Git Revert",  
  "timestamp_iso": "2026-10-01T08:04:58.168213+00:00",  
  "commits": {  
    "initial": "9a3468f11285c955a17e6980c57aeef63274a73",  
    "revert": "2d34aca6a4759d14acaf2d2af6c43ff1145ed189"  
  },  
  "traffic_during_bad_rollout": {  
    "total_requests": 200,  
    "successful_requests": 200,  
    "failed_requests": 0,  
    "error_rate_percent": 0.0  
  },  
  "recovery_time_seconds": 37.19  
}
```

Listing 4: Rohdatenprotokoll exp2-bad-deploy.json

A.3. Experiment 3: Konfigurationsdrift und Argo-CD-Self-Heal

Das Überwachungsskript skalierte das Deployment imperativ auf 0 Replikate und stoppte die Zeitspanne bis zur automatischen Selbstreparatur durch Argo CD:

```
python experiments/exp3_config_drift.py \  
--deployment resilience-lab \  
--namespace default \  
--app-name resilience-lab \  
--output experiments/results/exp3-config-drift.json
```

Listing 5: Ausführungsbefehl zu Experiment 3

```
{  
  "experiment": "Experiment 3 - Konfigurationsabweichung und Self-Heal",  
  "git_commit": "07a858dac10fe5e76bec13440a743e7a9331bd21",  
  "metrics": {  
    "detection_duration_seconds": 2.58,  
    "total_recovery_duration_seconds": 11.28,  
    "success": true  
  }  
}
```

Listing 6: Rohdatenprotokoll exp3-config-drift.json

Anhang B: Dokumentation des KI-Einsatzes und Promptverzeichnis

Gemäß den Vorgaben der Rheinischen Hochschule Köln zur guten wissenschaftlichen Praxis und Transparenz beim Einsatz von generativer künstlicher Intelligenz (KI) wird im Folgenden der genaue Umfang der KI-gestützten Vorarbeiten dargelegt.

B.1. Methodische Abgrenzung der Eigenleistung

Der Einsatz von generativen Sprachmodellen (Google Antigravity / LLM) beschränkte sich strikt auf Hilfs-, Beschleunigungs- und Syntax-Aufgaben:

- Erstellung der zustandslosen Demo-Webanwendung (Mock-API und HTML-Dashboard), da die reine Anwendungsentwicklung nicht den wissenschaftlichen Kern der Arbeit darstellt.
- Bereitstellung von Python-Messschleifen zur Latenzerfassung.
- Syntaxvalidierung von Konfigurationsdateien (GitHub Actions, Kustomize, Terraform).
- Typografische Unterstützung bei der Formatierung von LaTeX-Tabellen.

Die Konzeption, die Formulierung der Forschungsleitfrage, die architektonischen Entwurfsentscheidungen, die Durchführung der Störungsexperimente auf AWS sowie die kritische Interpretation der Ergebnisse stellen originäre Eigenleistungen des Autors dar.

B.2. Promptverzeichnis

Nachfolgend sind die zehn zentralen Prompts aufgeführt, mit denen die unterstützenden Komponenten generiert und formatiert wurden:

1. Prompt 1 (FastAPI Demo-Backend):

„Schreibe mir ein minimales Python-FastAPI-Backend für einen Kubernetes-Testserver. Ich brauche Standard-Routen für `/health/live`, `/health/ready`, `/api/info` (liefert Hostname, IP und Uptime) sowie einen POST-Endpunkt `/lab/crash`, der den Prozess hart via `os._exit(1)` beendet.“

2. Prompt 2 (Web-Dashboard UI):

„Erstelle mir eine einfache, responsive HTML-Seite mit modernem Dark-Mode-CSS für die FastAPI-App. Die Seite soll per JavaScript-Fetch alle 1,5 Sekunden `/api/info` abfragen und den antwortenden Pod-Namen sowie die Latenz anzeigen, damit man Pod-Wechsel visuell nachvollziehen kann. Keine externen Frameworks wie React oder Tailwind.“

3. Prompt 3 (Gehärtetes Dockerfile):

„Erstelle mir ein schlankes Dockerfile auf Basis von `python:3.11-slim` für diese FastAPI-App. Achte darauf, dass die App unter einem unprivilegierten Benutzer (`non-root user`) läuft und ein Standard-HEALTHCHECK definiert ist.“

4. Prompt 4 (Lastgenerator-Skript):

„Schreibe mir ein Python-Skript mit `concurrent.futures.ThreadPoolExecutor`, das kontinuierlich 15 parallele HTTP-GET-Requests pro Sekunde gegen eine URL sendet, HTTP-Statuscodes zählt und am Ende die Perzentile p_{50} und p_{95} der Antwortzeiten als JSON ausgibt.“

5. Prompt 5 (GitOps-Drift Automatisierung):

„Erstelle mir ein Python-Skript, das über `kubectl` den Befehl `scale deployment --replicas=0` absetzt und in einer Sekundenschleife den Status `status.sync.status` der Argo-CD-Application abfragt, um die exakte Zeitspanne bis zum Auftreten von `OutOfSync` sowie die Zeit bis zum Wiedererreichen von 2 Replikaten zu stoppen.“

6. Prompt 6 (GitHub Actions Workflow):

„Wie sieht die YAML-Syntax für eine GitHub-Actions-Pipeline aus, die bei einem Push auf `main` ein Docker-Image baut und sich mit dem automatischen `GITHUB_TOKEN` an der GitHub Container Registry (`ghcr.io`) anmeldet?“

7. Prompt 7 (Kustomize-Overlays):

„Gib mir die Kustomize-Syntax für ein Base-Deployment (1 Replikat) und ein Overlay `overlays/aws`, das per Patch die Replica-Zahl auf 2 erhöht und eine Traefik-Ingress-Ressource hinzufügt.“

8. Prompt 8 (AWS Security Group Parameter):

„Ich erhalte bei Terraform den Fehler: `InvalidParameterValue: Value for parameter GroupDescription is invalid. Character sets beyond ASCII are not supported.` Woran liegt das in meiner `security-groups.tf`?“

9. Prompt 9 (k3s TLS-SAN-Konfiguration):

„Mein lokales `kubectl` wirft `x509: certificate is valid for 10.0.1.88, not 63.176.27.134`, wenn ich mich mit der Elastic IP verbinde. Wie konfiguriere ich `tls-san` in k3s nachträglich, damit das Zertifikat die öffentliche IP akzeptiert?“

10. Prompt 10 (LaTeX-Tabellengestaltung):

„Ich habe folgende Messergebnisse aus einem Störungsversuch: Zeitpunkte $t_0 = 0$ s (Synced, 2 Pods), $t_2 = 2,6$ s (OutOfSync), $t_4 = 11,3$ s (Synced, 2 Pods). Formatiere mir diese Daten in sauberem LaTeX-Code unter Verwendung des `booktabs`-Pakets mit `\toprule`, `\midrule` und `\bottomrule` passend für eine DIN-A4-Projektarbeit.“

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen Publikationen, Vorlagen und Hilfsmittel (z.B. künstliche Intelligenz) als die angegebenen benutzt habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt für von mir verwendete Internetquellen. Ich versichere, dass ich diese Arbeit oder nicht zitierte Teile daraus vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Mir ist bekannt, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs mittels einer Plagiatserkennungssoftware auf eine ungekennzeichnete Übernahme von fremdem geistigen Eigentum sowie auf die Nutzung von künstlicher Intelligenz zur Texterstellung überprüft werden kann. Ich versichere, dass die elektronische Form meiner Arbeit mit der gedruckten Version identisch ist.

[Ort], 1. Oktober 2026

Ort, Datum

Unterschrift